

It's All Connected (Miras) — 한국어 의역본

It' s All Connected — 한국어의역본

이 문서는 arXiv:2504.13173 「It' s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization」 (Behrouz, Razaviyayn, Zhong, Mirrokni (Google Research)) 의 한국어 의역본입니다. 스타디용이며 공식 번역이 아닙니다. 수식 · 수치 · 그림은 원문기준이고, 그림은 원저자의 것입니다. 자연스러운 이해를 위해문장은 의역했습니다.

초록 (Abstract)

효율적이면서도 강력한 아키텍처 백본을 설계하는 일은 파운데이션 모델의 성능을 끌어올리려는 연구의 핵심에 놓여 있다. 이 논문은 인간의 인지현상인 **attentional bias**(특정 사건이나 자극을 우선적으로 처리하려는 자연스러운 경향)에서 영감을 얻어, Transformer·Titans·현대적 선형 순환신경망(linear RNN)을 포함한 여러 신경 아키텍처를 **키-값 매핑을 내부 목적함수로 학습하는 associative memory(연상 메모리) 모듈**로 다시 개념화한다. 이 내부 목적함수를 우리는 attentional bias라고 부른다. 흥미롭게도, 기존의 거의 모든 시퀀스 모델은 (1) 내적(dot-product) 유사도 또는 (2) ℓ_2 회귀(regression) 목적함수 중 하나를 attentional bias로 사용하고 있음을 관찰하였다. 이 두 목적함수를 넘어서, 우리는 학습과정을 안정화하는 근사 기법과 함께 대안적인 attentional bias 구성들을 제시한다. 나아가 현대 딥러닝 아키텍처의 forgetting(망각) 메커니즘을 **retention(보존) 정규화**의 한 형태로 재해석하여, 시퀀스 모델을 위한 새로운 forget gate 집합을 제안한다. 이러한 통찰을 바탕으로, 우리는 (i) associative memory 아키텍처, (ii) attentional bias 목적함수, (iii) retention gate, (iv) memory 학습 알고리즘이라는 네 가지 선택으로 딥러닝 아키텍처를 설계하는 일반 프레임워크 **Miras**를 제시한다. 그리고 세가지 새로운 시퀀스 모델 **Moneta, Yaad, Memora**를 제안하는데, 이들은 기존 linear RNN의 능력을 넘어서면서도 빠른 병렬 학습을 유지한다. 실험 결과 Miras 내의 서로 다른 설계 선택이 저마다 다른 강점을 갖는 모델을 낳음을 보였으며, 특정 인스턴스는 언어 모델링·상식추론·회상 집약적(recall-intensive) 과제에서 Transformer 및 현대 linear RNN을 능가하는 뛰어난 성능을 달성한다.

1 서론 (Introduction)

시퀀스 모델링을 위한 효율적 아키텍처 백본 설계는 언어 (Behrouz et al. 2024c; Vaswani et al. 2017a), 컴퓨터 비전 (Dosovitskiy et al. 2020), 계산 생물학 (Wang et al. 2024), 신경과학 (Behrouz et al. 2024a)에 이르기까지 파운데이션 모델의 성능을 높이는 핵심 열쇠이다. Transformer (Vaswani et al. 2017a)는 in-context learning 능력과 대규모 학습 능력 (Kaplan et al. 2020) 덕분에 시퀀스 모델링의 SOTA로 확고히 자리잡았지만, 시간·공간 복잡도가 제곱(quadratic)으로 증가하기 때문에 긴 문맥을 다루는 과제에서는 적용에 한계가 있다.

이런 한계를 극복하기 위해 최근 연구들은 효율적인 순환 대안을 설계해왔다. Transformer의 선형적으로 커지는 메모리(즉 KV 캐시)와 달리, 이 모델들은 문맥을 **고정 크기 메모리**로 압축하며, 그만큼 더 나은 메모리 관리가 요구된다. 더 효과적인 아키텍처를 위해 연구는 세 방향에서 메모리 용량과 관리를 개선해 왔다. (1) **학습 규칙**: Hebbian 규칙 (Hebb 2005)에서 Delta 규칙 (Neil et al. 2017)으로. (2) **Forget gate**: LSTM (Schmidhuber et al. 1997)에서 Mamba2 (Dao et al. 2024)로, 다시 Titans (Behrouz et al. 2024c)의 forget gate로. (3) **더 표현력 있는 메모리 아키텍처**: RetNet (Sun et al. 2023)·LRU (Orvieto et al. 2023)의 벡터 메모리에서 Titans·TTT (Sun et al. 2024)의 신경망 깊은 메모리로.

이러한 발전의 핵심에는 하나의 근본 질문이 있다. “**이 시퀀스 모델들 뒤에 있는 설계 프레임워크는 무엇이며, 이 모델들을 어떻게 더 강화할 수 있는가?**” 신경심리학의 associative memory와 학습 개념 (Okano et al. 2000)에서 영감을 받아, 여러 연구가 Transformer 및 (선형) RNN을 associative memory와 연결지어 왔다 (Bietti et al. 2023; Hopfield 1982; Ramsauer et

al. 2021). 그러나 이들은 (1) 밑바탕 학습 알고리즘을 온전히 설명하는 보편적 해석을 제공하지 못하거나, (2) 특정 정의에 국한되어 일반성이 부족하거나, (3) forget gate 처럼 표준적으로 널리 쓰이는 구성요소를 설명하지 못한다.

기여. 인간 인지 현상인 attentional bias 에서 영감을 받아, 우리는 Transformer·Titans·현대 linear RNN 을 **광의의 associative memory + attentional bias** 관점으로 다시 개념화한다. attentional bias 를 시퀀스 모델의 **내부 메모리 목적함수**로 정의·형식화하며 (3 장), 이 관점은 기존의 거의 모든 시퀀스 모델이 사실상 **같은 유형의 attentional bias** 를 쓰고있음을 드러낸다. 또한 기존 forgetting 메커니즘을 attentional bias 에 대한 **retention ℓ_2 정규화**로 재해석하고, 새로운 대안적 retention gate(forget gate) 집합을 제안하여 “새 개념 학습” 과 “기존 개념 보존” 의 균형에 관한 새로운 통찰을 제공한다.

이 형식화 위에서 우리는 **Miras** 를 제시한다.¹ Miras 는 (1) attentional bias(메모리 목적함수), (2) retention gate, (3) memory 아키텍처, (4) memory 학습 알고리즘 (optimizer) 이라는 네가지 선택으로 새로운 시퀀스 아키텍처를 설계하는 근본 프레임워크이며, 우리는 여러 새로운 설계 선택을 동기와 함께 논의한다.

마지막으로 우리는 Miras 의 세 변형 **Moneta, Yaad, Memora** 에 집중한다. 이들은 단순한 ℓ_2 회귀를 넘어선 attentional bias 와, 기존보다 더 견고한 (robust) retention 게이팅 메커니즘에 기반한다. 언어모델링·상식추론·needle-in-haystack·회상 집약적 과제에 대한 실험은 이 변형들이 SOTA 시퀀스 모델을 능가함을 보인다.

로드맵. 2 장에서 관련 연구와 배경 개념을 정리한다. 3 장에서 associative memory 의 광의 정의와 attentional bias 개념을 형식적으로 도입하고, 시퀀스 모델링을 최적화 관점에서 해석하는 두 시각 — **Learning-Retaining** 과 **Follow-the-Regularized-Leader(FTRL)** — 을 논의하며 전자가 후자를 포괄함을 증명한다. 4 장에서 Miras 프레임워크를 제시하고 현대 시퀀스 모델들이 어떻게 통합되는지 보인다. 5 장에서 attentional bias 와 retention gate 에 대한 새로운 설계 선택을 논의하고, 5.3 절에서 세 변형을 제시한 뒤병렬 학습법을 설명한다. 6 장에서 실험 결과를 보고한다.

2 예비 개념과 배경 (Preliminaries and Background)

이 절에서는 논문 전반에서 사용하는 관련 연구와 배경 개념을 정리한다.

Attention. Transformer 의 백본인 attention 은 그 associative memory 역할을 하는 핵심 구성요소이다 (Bietti et al. 2023). 입력 $x \in \mathbb{R}^{N \times d_{in}}$ 에 대해 causal attention 은 입력의존적인 key·value·query 행렬에 대한 Softmax 로출력 $y \in \mathbb{R}^{N \times d_{in}}$ 을 계산한다:

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V,$$

$$y_i = \frac{\sum_{j=1}^i \exp(q_i^\top k_j / \sqrt{d_{in}}) v_j}{\sum_{\ell=1}^i \exp(q_i^\top k_\ell / \sqrt{d_{in}})}.$$

여기서 $W_Q, W_K, W_V \in \mathbb{R}^{d_{in} \times d_{in}}$ 은 학습파라미터이다. Transformer 는 LSTM 같은 전통 RNN 대비 큰 개선을 이뤘으나, 출력 계산에 최소 $N \times d$ 연산이 필요한 복잡도가 대안 아키텍처연구를 자

¹ “Miras” 는 페르시아어·아랍어·터키어 등 여러 언어에서 “유산 (Legacy)” 을 뜻한다. 이 프레임워크가 이후 연구가 과제에 맞는 강력한 시퀀스 모델을 설계할 명확한 발판을 제공한다는 뜻에서 이 이름을 택했다.

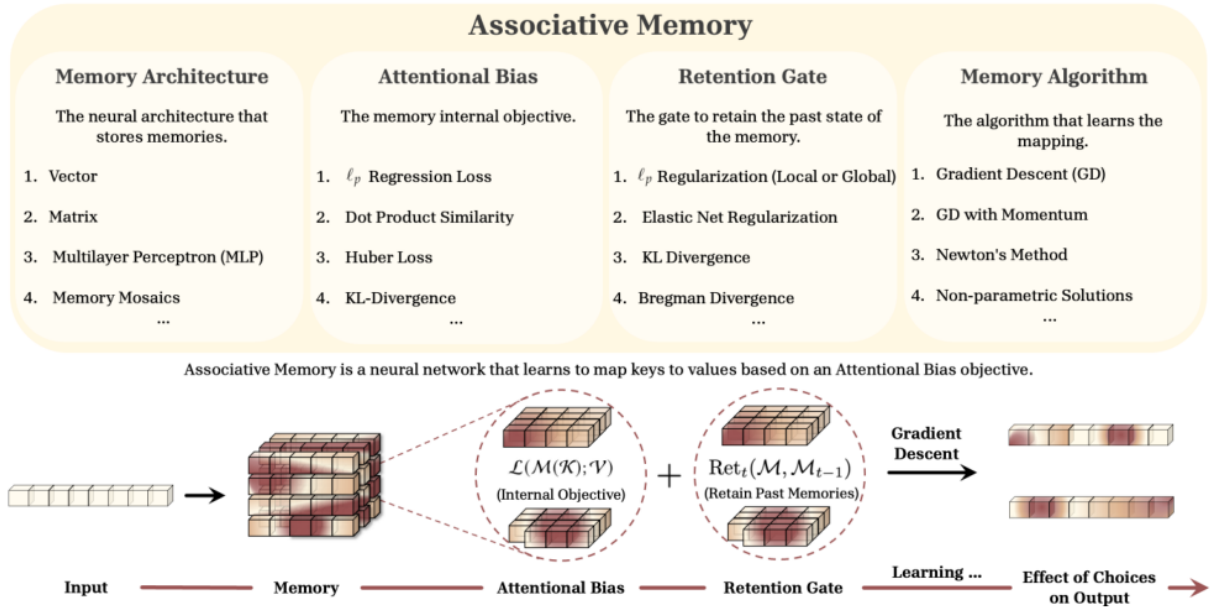


그림 1: Miras 프레임워크 개요. Miras 는 (1) 메모리 아키텍처, (2) attentional bias, (3) retention gate, (4) memory 학습 알고리즘이라는 네가지 핵심 선택에 기반한다. 메모리 아키텍처는 기억 용량을, attentional bias 는 밀바탕 매핑 패턴 모델링을, retention gate 는 새 개념 학습과 기존개념 보존 사이의 균형을, memory 학습 알고리즘은 메모리 관리를 담당한다. 출처: 원저자, arXiv:2504.13173 Fig.1

극한 주된 동기였다. 우리는 이 대안 연구를 (1) 얕은메모리 (shallow memory) 선형 순환 모델과 (2) 깊은 메모리 (deep memory) 모듈의 두 갈래로 나누어 정리한다.

(선형) 순환 모델. 오랫동안 비선형 (게이트) RNN 이 딥러닝의 사실상표준 백본이었으나 (Greff et al. 2016), 그 순환적 성질 탓에 학습을병렬화할 수 없어 대규모 학습이 어려웠다. 이에 최근 몇 년 간 **linear RNN** 이 병렬화 가능 · 선형 시간 학습이라는 이점덕에 주목받고 있다 (Peng et al. 2025a; Sun et al. 2023; Yang et al. 2024c). 초기 linear RNN(De et al. 2024; Sun et al. 2023; Yang et al. 2024b) 은 대부분 Hebbian 학습 규칙 (Hebb 2005) 에 기반해 데이터를벡터형 (또는 행렬형) 메모리로 압축한다. $M_t \in \mathbb{R}^{d \times n}$ 을 메모리 ($n = 1$ 이면 벡터형), $k, v \in \mathbb{R}^d$ 을 입력 x_t 의 투영으로 얻은 key-value 라 할 때, 이런 linear RNN 의 일반식은 다음과 같다:

$$M_t = A_t * M_{t-1} + v_t k_t^\top,$$

여기서 $*$ 는 임의의 결합적 (associative) 연산자이고 A_t 는 데이터 (비) 의존적 대각 행렬 또는 스칼라이다 (Yang et al. 2024c). 이런 모델은효율적이지만, Hebbian 규칙의 **가산적 (대체 없는) 갱신** 성질 때문에메모리가 overflow 되어 용량과 in-context 학습 표현력이 제한되며, 벡터형메모리는 긴 문맥의 밀바탕 패턴을 학습하는 표현력이 부족하다.

이를 해결하기 위해 **Delta 학습 규칙**을 쓰는 행렬형 메모리 순환모델이 인기를 얻었다 (Neil et al. 2017; Schlag et al. 2021; Yang et al. 2024c). 큰 장점에도 불구하고, delta-rule 기반 모델조차 이론적한계 (Irie et al. 2023) 와 실무에서의 중간 수준 성능 (Yang et al. 2024c) 을보인다. 최근 연구는 스칼라/채널별 forget gate(Peng et al. 2025b; Yang et al. 2024a), 음의 고ؤ값 (Grazzi et al. 2024), 다단계학습 (Siems et al. 2025) 으로 이를 개선하려 하지만, 여전히 표현력이 낮은메모리 아키텍처 탓에 긴 문맥에서 성능이 떨어진다.

깊은 메모리 모듈: Titans 와 Test-Time Training. 제한된 메모리를극복하고 유효 문맥 길이를 늘리기 위해, 최근 연구는 **깊은 메모리모듈**을 갖춘 새로운 세대의 아키텍처에 집중한다 (Behrouz

et al. 2024c; Sun et al. 2024). 이 아키텍처는 메타러닝 관점 위에 서 있는데, 메모리가 MLP 이고 이를 (모멘텀을 곱들인) 경사하강으로 갱신한다. Sun et al.(2024) 은 선형 attention 과 softmax attention 이 각각 (커널) 회귀 손실의 **파라메트릭 · 비파라메트릭** 해임을 밝히는 통합 관점을 제공했으나, 다른 현대 linear RNN 은 이 클래스 밖으로 두었다. 우리와 동시기 연구인 Wang et al.(2025) 은 현대 RNN(RetNet, Mamba 등) 을 추가로 단순화하면 회귀 손실을 내부적으로 최적화하는 같은 클래스에 근사적으로 놓임을 보였다. 그럼에도 “**기존 아키텍처를 정확히 통합할 수 있는 설계 프레임워크는 무엇인가?**” 라는 질문은 여전히 미해결이며, forget gate 와 그 대안적 선택의 역할은 놀랍도록 덜 탐구되어 있다.

3 Associative Memory, Attentional Bias, Retention

associative memory 는 인간 학습에서 떨 수 없는 요소 (Terry 2017) 로, 많은 인공 신경 아키텍처의 영감이 되어 왔다 (Behrouz et al. 2024c; Hopfield 1982; Neil et al. 2017). 그러나 기존 연구는 associative memory 의 특정 인스턴스만 정의하여, 아키텍처를 (키 · 값 간) 특정 유사도 척도를 래스로 국한시켰다. 넓게 보면 associative memory 는 키 집합 K 를 값 집합 V 로 매핑하는 연산자이며, 데이터의 밑바탕 매핑 패턴을 학습하려면 메모리의 유형을 규정하고 학습된 매핑의 질을 재는 목적함수가 필요하다.

정의 3.1 (Associative Memory 와 Attentional Bias). 키 집합 $K \subseteq \mathbb{R}^{d_k}$ 과 값 집합 $V \subseteq \mathbb{R}^{d_v}$ 이 주어질 때, associative memory 는 연산자 $\mathcal{M} : K \rightarrow V$ 이다. 이 매핑의 학습은 목적함수 L — 이를 **Attentional Bias** 라 부르며, 메모리의 유형과 어떤 사건을 우선할지의 경향을 규정한다 — 을 최소화하는 것으로 이루어진다:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} L(\mathcal{M}(K); V).$$

이 식은, 메모리가 좋은 recall 이라 여기는 기준 (=attentional bias) 을 정하고 그에 따라 학습된 매핑을 최적화한다는 뜻이다. 몇 가지 주석을 덧붙인다.

주석 1. 메모리를 파라미터 W 로 파라미터화하면 $\mathcal{M}(W, k)$ 로 쓰며, 이 경우 위 최적화는 W 에 대해 수행된다. 또한 파라메트릭 설정에서는 과거 데이터의 retention 을 제어하기 위해 추가 정규화 $R(W)$ 를 쓸 수 있다.

주석 2. 키-값 매핑 학습은 **메타러닝 문제**이다. attentional bias 는 **inner-loop** 에서 최적화되고, 신경망의 다른 모든 파라미터 (선형 투영, 컨볼루션 등) 는 **outer-loop** 에서 최적화된다. 즉 모델은 **테스트 시점**에 데이터를 자기 파라미터에 저장하는 법을 배운다 (Behrouz et al. 2024c; Sun et al. 2024).

3.1 최적화의 렌즈로 본 기억과 보존

정의 3.1 은 신경 아키텍처 설계를 목적함수 L 의 최소화로 번역한다. 이를 최적화하는 간단한 방법은 경사하강이다. 새 키-값 쌍이 들어오면 메모리를 다음과 같이 갱신한다:

$$W_t = W_{t-1} - \eta_t \nabla \ell(W_{t-1}; k_t, v_t),$$

여기서 편의상 $\ell(W_{t-1}; k_t, v_t) := L(\mathcal{M}(W; k_t), v_t)$ 로 둔다. 이 식은 새 토큰에 대해 목적함수의 그라디언트만큼 메모리를 한 발짝 갱신한다는 뜻이다. Behrouz et al.(2024c) 은 이 양을 **순간적 놀람 (momentary surprise)** 지표로 재해석했는데, 목적함수의 기대를 위반하는 (즉 메모리에 놀라운) 토큰을 기억한다는 것이다. 목적함수의 선택은 이 식을 온전히 해석하는 데 중요한 단계지만 (5장에서 상세히 다룸), 이 갱신 규칙을 일반형으로 해석하는 서로 다른 시각들이 있고 이는 기존 아키텍처를 넘어서게 도와준다.

3.2 시각 1: 온라인 회귀와 Follow-The-Regularized-Leader

위 갱신은 손실 함수열 $\ell(W; k_1, v_1), \ell(W; k_2, v_2), \dots, \ell(W; k_t, v_t), \dots$ 에대한 온라인 경사하강 한 발짝으로 볼 수 있다. 온라인 경사하강은 특정 손실 선택 하에서 **FTRL(Follow-The-Regularized-Leader)** 의 특수 사례임이 잘 알려져 있다 (Shalev-Shwartz et al. 2012, ch.2; Hazan et al. 2016). 구체적으로 $W_0 = 0$ 이라 가정하면 위 갱신은 다음과 동치이다:

$$W_t = \arg \min_W \sum_{i=1}^t \langle W - W_{i-1}, \nabla \ell(W_{i-1}; k_i, v_i) \rangle + \frac{1}{2\eta} \|W\|_2^2.$$

여기서 $\langle W - W_{i-1}, \nabla \ell(W_{i-1}; k_i, v_i) \rangle$ 는 시각 i 에서 원 손실의 국소 선형 근사이고, 둘째 항은 정규화이다. 첫째 항은 메모리가 모든 과거 토큰을 얼마나 잘 학습하는지를 재고, 둘째 항 $\frac{1}{2\eta} \|W\|_2^2$ 은 메모리의 크기에 대해 갱신을 벌한다. 이 식은 손실의 선형 근사와 이차 정규화를 쓰지만, 원칙적으로 다른 근사와 다른 정규화 함수를 쓸 수 있으며 이것이 mirror descent 같은 알고리즘의 발상이다. 이를 일반화하면:

$$W_t = \arg \min_{W \in \mathcal{W}} \underbrace{\sum_{i=1}^t \hat{\ell}_i(W; k_i, v_i)}_{\text{Attentional Bias}} + \underbrace{\frac{1}{\eta_t} R_t(W)}_{\text{Memory Stability}}. \quad (\text{FTRL Viewpoint})$$

이 식은, 합 항 $\sum_i \hat{\ell}_i$ 이 테스트 시점에 토큰을 기억하는 역할을, $R_t(W)$ 가 새 데이터로 갱신할 때 메모리 크기를 고려해 학습동역학을 정규화하는 역할을 한다는 뜻이다. $\hat{\ell}_i$ 과 $\frac{1}{\eta_t} R_t(W)$ 의 선택에 따라 (온라인) 경사하강, mirror descent 등 다른 알고리즘이 나온다. 이 일반화에서 η_t 는 데이터의 존적일 수 있고, $\mathcal{W}$ 로 W 에 제약을 부과할 수도 있다.

3.3 시각 2: 최신 토큰을 학습하며 이전 정보를 보존

같은 갱신을 해석하는 또 다른 방법은, 최신 키-값 쌍 (k_t, v_t) 로부터 (그래디언트/놀람 지표를 통해) 학습하면서 이전 상태 W_{t-1} 에 가깝게 머물러 이미 기억한 토큰을 보존하는 것으로 보는 것이다. 형식적으로:

$$W_t = \arg \min_W \langle W - W_{t-1}, \nabla \ell(W_{t-1}; k_t, v_t) \rangle + \frac{1}{2\eta_t} \|W - W_{t-1}\|_2^2.$$

첫째 항은 $\ell(W; k_t, v_t)$ 을 W_{t-1} 근방에서 국소 근사하고, 마지막 항은 W_{t-1} 로부터의 이탈을 정규화한다. 이를 일반화하면:

$$W_t = \arg \min_{W \in \mathcal{W}} \underbrace{\tilde{\ell}_t(W; k_t, v_t)}_{\text{Attentional Bias}} + \underbrace{\text{Ret}_t(W, W_{t-1})}_{\text{Retention}}. \quad (\text{Learning-Retaining Viewpoint})$$

$\tilde{\ell}_t$ 을 최소화하는 것은 새 개념 (k_t, v_t) 로부터 학습하는 것에, Ret_t 은 W 의 변화를 정규화해 학습을 안정화하고 이전 지식을 보존하는 것에 해당한다. 이 retention 함수는 국소 · 전역 성분으로 분해될 수 있다:

$$\text{Ret}_t(W, W_{t-1}) = \underbrace{\frac{1}{\eta_t} D_t(W, W_{t-1})}_{\text{Local Retention}} + \underbrace{\frac{1}{\alpha_t} G_t(W)}_{\text{Global Retention}} .$$

이 식은, D_t 가 W_{t-1} 로부터의 이탈을 제어하는 premetric 으로 이전 지식 보존을 담당하고, G_t 가 메모리 크기에 대한 변화를 제어하는 전역 retention 임을 뜻한다. 계수 η_t 는 **메타 in-context 학습률**로 볼 수 있는데, η_t 가 클수록 새 개념을 더 많이 학습하고 이전 개념을 더 많이 잊는다. 이 시각의 특수 사례 (전역 retention 이 없거나, 암시적 닫힌 해를 갖거나, 제한된 메모리 구조인 경우) 가 Liu et al.(2024a) 등의 동기가 되었다.

3.4 두 시각에 대한 추가논의

(FTRL Viewpoint) 과 (Learning-Retaining Viewpoint) 은 온라인 최적화의 렌즈로 연결된다. 예컨대 손실의 선형 근사와 이차정규화/retention 을 택하면 둘 다 위의 온라인 경사하강을 특수 사례로 포괄한다. 이 연결을 명시하는 한 방법은 premetric $D_t(W; W')$ 을과거 손실과 정규화로 정의하는 것이다:

명제 3.2. $\eta_t = \eta$ 이라 하고 $h_t(W) := \sum_{i=1}^{t-1} \hat{\ell}_i(W; k_i, v_i) + \frac{1}{\eta} R(W)$ 로 정의하자. $\mathcal{W} = \mathbb{R}^d$ 이고 $h_t(W)$ 가 W 에 대해강볼록 (strictly convex) 이라 가정하고, $D_h(\cdot, \cdot)$ 을 h 가 정의하는 Bregman divergence, 즉 $D_h(W, W') = h(W) - h(W') - \langle \nabla h(W'), W - W' \rangle$ 이라 하자. (Learning-Retaining Viewpoint) 에서 $\text{Ret}_t(W, W') = D_h(W, W')$ 과 $\tilde{\ell}_t(W; x_t) = \hat{\ell}_t(W; x_t)$ 로 두면, (Learning-Retaining Viewpoint) 의 갱신 규칙은 (FTRL Viewpoint) 의 갱신 규칙과 동치이다.

증명은 부록 B 에 있다. 이 명제는 (Learning-Retaining Viewpoint) 이 온화한 가정 하에서 (FTRL Viewpoint) 으로 얻는 접근들도 설명할 수 있음을, 즉 더 일반적인 시각임을 보인다. 이것이 이후 유도에서 이 시각을 주로 쓰는 이유다.

주석 3. 이 시각에서는, 추가적 전역 정규화를 쓰더라도 실제로 메모리를 지우거나 잊는 과정은 없으며, 다만 모델이 이전 상태를 보존하지 않기로 결정할 뿐이다. 흥미롭게도 이는 인간 기억과정과도 맞아떨어진다 — 뇌는 기억을 지우지 않고, 다만 인출실패 (retrieval failure) 로 접근 불가능해질 뿐이다 (Robertson 2002). 따라서 이 항을 forget gate 대신 **Retention Gate** 라 부른다.

주석 4. 4 장과 표 1 에서 요약하듯, 대부분의 현대 시퀀스 모델은 attentional bias 를 경사하강으로 최적화한다. 위 두 시각은 이해를 돕기 위해 경사하강 기반 갱신에 국한해 논의했지만, 정의 3.1 의 attentional bias 와 associative memory 는 더 넓으며 어떤 최적화 알고리즘으로도 (뉴턴 법이나 비파라메트릭 해 포함) 최적화될 수 있다.

4 Miras: 견고하고 표현력 있는 메모리로 기억하기를 학습

associative memory, attentional bias, 두 시각의 정의 위에서, 우리는 기존 백본 아키텍처를 정확히 통합하면서 차세대 시퀀스 모델 설계에 대한 통찰도 주는 **Miras 프레임워크**를 제시한다. associative memory 학습은 메타러닝 과제로 해석되며, 메모리는 테스트 시점에 데이터를 압축 · 저장하는 법을 배운다. 긴 문맥에서는 메모리 구조의 표현력이 학습 능력을 제한하므로, 첫 설계 선택은 메모리 구조다. 구조가 정해지면 파라미터 W 에 대해 손실 $\ell(W; \cdot, \cdot)$ 을 retention 정규화 $\text{Ret}(\cdot)$ 과 함께 학습 알고리즘으로 최소화한다. 요컨대 Miras 는 네 가지 설계 선택을 요구한다:

1. **메모리 구조 (Memory Structure).** 메모리의 아키텍처를 정한다. 벡터, 선형 함수, MLP, 혹은 더 복잡한 구조가 가능하다. W 를 특정영역 (예: L_2 공 안) 으로 제한해 무한값 · 불안정 학습을 피할 수도 있다.
2. **Attentional Bias.** 정의 3.1 의 목적함수 $L(\cdot)$ 이다. 그근사 ($\hat{\ell}$ 또는 $\tilde{\ell}$) 도 고려할 수 있다. 이 선택은 메모리가 문맥을 어떻게 기억하고, 입력을 어떻게 매핑하며, 어떤 사건을 우선할지를 정한다.
3. **메모리 안정성과 Retention.** retention 정규화 $R(\cdot)$ (FTRL 의 R_t 또는 Learning-Retaining 의 Ret_t) 이다. 파라메트릭 설정에서 학습과 과거 상태 보존의 균형을 잡는다. **효과적인 retention gate 가 긴 문맥 성능의 핵심**이다.
4. **메모리 알고리즘 (Memory Algorithm).** 목적함수를 최적화하는 학습 알고리즘이다. 경사 하강, 모멘텀 경사하강, 그 밖의 알고리즘 (비파라메트릭 해 포함) 을 쓸 수 있다.

이 네 가지가 주요 설계 선택이며, 그 외에 모델을 구별하는 부차적 결정 — 데이터 (비) 의존적 파라미터, 스칼라 vs 채널별 학습률/retention gate 등 — 이 있다. 다음으로 기존 아키텍처가 Miras 에 어떻게 들어맞는지 개관한다.

표 1: Miras 관점에서 본 최근 시퀀스 모델 개요. 놀랍게도 모든 모델이 같은 유형의 **attentional bias** 와 **정규화 (forget gate)** 를 쓴다. 단, 이 설계 선택이 백본을 유일하게 식별하지는 않는다 (입력 의존성, 채널별 파라미터, attention · 컨볼루션 등 다른 요소가 있다). **attentional bias** 와 **retention gate** 는 각 모델의 **원 설계** 기준이다.

모델	메모리 구조	Attentional Bias	Retention Gate †	메모리 알고리즘
얕은 메모리 (Shallow Memory)				
RetNet (2023)	Vector	Dot-Product	L2	GD
Transformer (2017)	Matrix	L2	–	Nonparametric
LA (2021)	Matrix	Dot-Product	L2	GD
DFW	Matrix	Dot-Product	L2	GD
Lightning Attention (2025)	Matrix	Dot-Product	L2	GD
GLA (2024)	Matrix	Dot-Product	L2	GD
Mamba (2024)	Matrix	Dot-Product	L2	GD
HGRN2 (2024)	Matrix	L1	–	GD
DeltaNet (2017)	Matrix	L2	–	GD
Longhorn (2024)	Matrix	L2	–	Implicit GD
TTT-Linear (2024)	Matrix	L2	–	GD
Gated DeltaNet (2024)	Matrix	L2	L2	GD
RWKV-7 (2025)	Matrix	L2	L2	GD
DeltaProduct (2025)	Matrix	L2	L2	MGD*
깊은 메모리 (Deep Memory)				
TTT-MLP (2024)	2-layer MLP	L2	–	GD
Titans-LMM (2024)	k -layer MLP	L2	L2	GD + Momentum
Moneta (ours)	2-layer MLP	L_p	L_q	GD
Yaad (ours)	2-layer MLP	Huber	L2	GD
Memora (ours)	2-layer MLP	L2	KL	GD

MGD = 토큰당 다중 GD 라운드. † 명료함을 위해 L2-계열 정규화를 모두 “L2” 로 표기했으나, 실제로 정확한 L2 retention gate 를 쓰는 것은 Titans 와 RWKV-7 뿐이다 (4 장 참조).

대표적인 메모리 쓰기 연산 (원문 표 1) 을 옮기면 다음과 같다. 얇은 메모리계열에서 RetNet 은 $M_t = \alpha M_{t-1} + v_t k_t^\top$, Transformer 는 $M_t = M_{t-1} \cup \{(k_t, v_t)\}$, GLA 는 $M_t = \text{Diag}(\alpha_t) M_{t-1} + v_t k_t^\top$, Mamba 는 $M_t = \alpha_t M_{t-1} + v_t k_t^\top$, HGRN2 는 $M_t = \text{Diag}(\alpha_t) M_{t-1} + v_t (1 - \alpha_t)^\top$, DeltaNet 은 $M_t = (I - \beta_t k_t k_t^\top) M_{t-1} + \beta_t v_t k_t^\top$, Gated DeltaNet 은 $M_t = \alpha_t (I - \beta_t k_t k_t^\top) M_{t-1} + \beta_t v_t k_t^\top$, RWKV-7 은 $M_t = \text{diag}(\alpha_t) (I - \beta_t k_t k_t^\top) M_{t-1} + \beta_t v_t k_t^\top$ 이다. 깊은 메모리 계열에서 TTT 는 $M_t = M_{t-1} - \eta \nabla L(M_{t-1}; k_t, v_t)$, Titans-LMM 은 $M_t = \alpha_t M_{t-1} - S_t$, $S_t = \eta_t S_{t-1} - \theta_t \nabla L(M_{t-1}; k_t, v_t)$ 이다.

Hebbian 규칙 RNN. 현대 순환 아키텍처의 첫 세대 (Linear attention, RetNet, Mamba, GLA) 는 (게이트) Hebbian 규칙에 기반한다. attentional bias 를 **내적 유사도**로 두어, 메모리 $\mathcal{M} \in \mathbb{R}^{d \times n}$ 과 $k, v \in \mathbb{R}^d$ 에 대해 $\tilde{\ell}_t := -2\langle \mathcal{M}_t k_t, v_t \rangle$, 국소 retention 을 $\text{Ret}_t(\mathcal{M}, \mathcal{M}_{t-1}) = \|\mathcal{M}_t - \alpha \mathcal{M}_{t-1}\|_F^2$ 로둔다. (Learning-Retaining Viewpoint) 에 경사하강을 적용하면:

$$\mathcal{M}_t = \alpha \mathcal{M}_{t-1} + v_t k_t^\top.$$

이 식은, (1) $\alpha = 1$ 이면 Linear Attention(LA), (2) $\alpha \in \mathbb{R}$ 이 학습 파라미터면 Lightning Attention($n > 1$) 또는 RetNet($n = 1$), (3) $\alpha_t \in \mathbb{R}$ 이 데이터 의존적 학습 파라미터면 Mamba2 가됨을 뜻한다.

Delta 규칙 RNN. 메모리 관리와 용량을 높이기 위해 여러 연구가 **delta** 규칙을 학습 알고리즘으로 제안했다 (DeltaNet, Longhorn, RWKV-7). $\mathcal{M} \in \mathbb{R}^{d \times n}$ 에서 delta 규칙은 MSE 목적함수 $\|\mathcal{M}_t k_t - v_t\|_2^2$ 을 국소 retention $\text{Ret}_t = \|\mathcal{M}_t - \alpha \mathcal{M}_{t-1}\|_F^2$ 과함께 확률적 경사하강으로 최적화하는 것과 동치이다:

$$\mathcal{M}_t = \alpha (I - \eta_t k_t k_t^\top) \mathcal{M}_{t-1} + v_t k_t^\top.$$

이 식은, (1) $\alpha = 1$ 이면 DeltaNet, (2) $\alpha_t \in \mathbb{R}^m$ 이 데이터 의존적이면 Gated DeltaNet($m = 1$) 또는 RWKV-7($m = d$) 이 됨을 뜻한다. 따라서 delta 규칙 RNN 도 Miras 의 특수 사례다.

Delta 규칙을 넘어서. delta 규칙은 값 대체 전략으로 Hebbian 보다 강력하지만 이론적 한계 (Irie et al. 2023) 와 중간 성능 (Yang et al. 2024c) 이 있다. 최근 **Titans**(Behrouz et al. 2024c) 는 비선형 MSE 목적함수 $\|\mathcal{M}_t(k_t) - v_t\|_2^2$ 을, 국소 retention $D_t = \|W_t - W_{t-1}\|_F^2$ 과 전역 retention $G_t = \|W_t\|_2^2$ 을함께 두고, **모멘텀 경사하강**으로 최적화한다.² 따라서 Titans-LMM 은 위 attentional bias·retention·모멘텀 경사하강을 쓰는 Miras 의특수 사례다. 또 다른 예 **Mesa-layer** 는 $\sum_{i=1}^t \|\mathcal{M}_t(k_i) - v_i\|_2^2$ 을 attentional bias, $\|\mathcal{M}_t\|_2^2$ 을 retention 으로 두고 **뉴턴법**으로 최적화하여 delta 규칙보다 표현력 있는 갱신을 제공한다.

Attention. Sun et al.(2024) 이 논한 대로, **softmax attention** 은 ℓ_2 -MSE 손실 $\|Wk - v\|_2^2$ 의 **Nadaraya-Watson 추정량에 의한 비파라메트릭 해**이다. 따라서 softmax attention 은, retention 없이 MSE 손실의 비파라메트릭 해를 찾을 때 나오는 Miras 의인스턴스이다. 이것이 attention 이 완벽한 recall 과 선형 상태 증가를 갖는이유다 — 압축을 전혀 하지 않기 때문이다.

5 기존 Attentional Bias 와 Retention Gate 를넘어서

앞 장에서 보았듯 기존 연구는 attentional bias·retention gate 에 선형/이차 선택만 쓴다. 구체적으로 손실 $L(\mathcal{M}(k_t), v_t)$ 은 학습 상수 c_t 에 대해 $c_t \|\mathcal{M}(k_t) - v_t\|^2$ 형태로, 정규화 $R_t(W)$ 나 D_t

²Titans 의 retention gate(forget gate) 는 Mamba2·Gated DeltaNet 과다르다. 차이는 **완전 삭제 (full erase)**의 경우에 있다. Mamba2 게이팅은 메모리 전체를 지우고 다음 토큰을 사상 처음 보는 데이터처럼 다루는 반면, Titans 는 “**콜드 스타트 (cold start)**” 전략으로메모리를 완전히 지우기 전에 이전 상태를 써서 들어오는 토큰의 놀람정도를 먼저 잰다.

는 이차/선형으로 두며, 거의 모든 연구가 W 를 전체 $\mathbb{R}^d$ 공간으로 둔다. 그러나 일반적으로 이 설계 선택 모두다양한 값을 취할 수 있다. 프레임워크의 잠재력과 유연성을 보이기 위해, 여기서 attentional bias 와 retention gate 에 대한 몇 가지 새로운 설계선택을 검토한다. 명료함을 위해 경사하강을 optimizer 로 삼아 논의하지만, 이 목적함수 · 정규화는 정의 3.1 에 직접 넣어 **어떤최적화 알고리즘으로도** 최적화할 수 있으며 그때마다 다른 갱신 규칙이 나온다.

5.1 대안적 Attentional Bias

변형 1: ℓ_p -Attentional Bias. attentional bias 는 “유사도척도” 를 정의하고, 키가 주어졌을 때 메모리가 값을 얼마나 잘 recall 하는지를 잴다. ℓ_2 회귀 손실이 자연스러운 선택이지만 데이터의 잡음에 민감하다. 자연스러운 확장은 ℓ_p -노름 목적함수다. 메모리 $\mathcal{M}$, 키 k , 값 v 에 대해:

$$L(\mathcal{M}(W, k_t); v_t) = \|\mathcal{M}(k_t) - v_t\|_p^p, \quad p \geq 1.$$

행렬 메모리 $W \in \mathbb{R}^{m \times d}$, $\mathcal{M}(W, k_t) = Wk_t$ 일 때 경사하강 갱신은:

$$W_t = W_{t-1} - p \eta_t [\text{Sign}(Wk_t - v_t) \odot |Wk_t - v_t|^{p-1}] k_t^\top.$$

이 식은 p 가 잡음 민감도를 조율함을 뜻한다 ($p < 2$ 는 견고, $p > 2$ 는 큰 오차를 강조). $p = 1$ 이면:

$$W_t = W_{t-1} - \eta_t \text{Sign}(W_t k_t - v_t) k_t^\top,$$

즉 메모리가 -1 과 $+1$ 두 값만 갖는다. 우리는 이를 **value-less associative memory** 라 부른다 — 키 (엔티티) 는 저장하되 그 값을 ± 1 의 두 극단 클래스로만 매핑한다.

주석 5. 위 갱신의 핵심 난점은 역전파 (backpropagation) 이다. $\text{Sign}(\cdot)$ 과 $|\cdot|$ 은 미분 불가능하여 학습을 불안정하게 만들 수 있다. 이를 해결하기 위해 $\text{Sign}(x) \approx \tanh(\alpha x)$, $|x| \approx \sqrt{x^2 + \epsilon}$ 을 매끄러운 근사로 쓴다.

value-less 메모리에 대한 한 해석은 인간의 **coping mechanism** (Loftus 1993) 과 유사하다 — 극단적 사건의 값을 저장하지 않는 방식이다. 이 보호적 기억이라는 해석이 다음 변형의 동기다.

변형 2: Huber 손실 — coping mechanism 을 갖춘 메모리. ℓ_2 목적함수는 이상치 (outlier) 와 극단 표본에 민감하다. 견고 회귀 문헌에서 동기를 얻어, 이상치가 메모리 학습에 미치는 부정적 영향을 줄이는 **Huber 손실** (Hastie et al. 2009; Huber 1992) 을 attentional bias 로 제안한다. 세 가지 방식이 있다.

(i) 좌표별 Huber 손실의 합:

$$\ell(W; k_t, v_t) = \sum_j \mathcal{H}(\mathcal{M}(W, k_t)_j - v_{t,j}), \quad \mathcal{H}(a) = \begin{cases} \frac{1}{2}a^2 & |a| \leq \delta, \\ \delta|a| - \frac{1}{2}\delta^2 & |a| > \delta. \end{cases}$$

행렬 메모리 $\mathcal{M}(W, k_t) = Wk_t$ 의 갱신은:

$$W_t = W_{t-1} - \eta_t \left[(Wk_t - v_t) k_t^\top \odot \mathbb{I}(|Wk_t - v_t| \leq \delta_t) \mathbf{1}^\top + \delta_t \text{Sign}(Wk_t - v_t) k_t^\top \odot \mathbb{I}(|Wk_t - v_t| > \delta_t) \mathbf{1}^\top \right].$$

이 식은, 파라미터 δ_t 가 문맥에 따라 메모리 블록마다 어떤유형 (ℓ_2 목적함수 또는 value-less) 을 쓸지 정해 이상치에 견고하게만든다는 뜻이다.

(ii) 잔차 노름에 대한 Huber, $\ell(W; k_t, v_t) = \mathcal{H}(\|\mathcal{M}(W, k_t) - v_t\|_2)$. 갱신은:

$$W_t = W_{t-1} - \eta_t \begin{cases} (\mathcal{M}(W_{t-1}, k_t) - v_t) k_t^\top & \|\mathcal{M}(W_{t-1}, k_t) - v_t\|_2 \leq \delta_t, \\ \delta_t \frac{\mathcal{M}(W_{t-1}, k_t) - v_t}{\|\mathcal{M}(W_{t-1}, k_t) - v_t\|_2} k_t^\top & \text{그 외.} \end{cases}$$

이 식은 임계값 이내면 원 잔차 외적을, 초과하면 정규화된 잔차 방향에 δ_t 를 곱한 것을 쓰는 **토큰별 그래디언트 클리핑**이다.

(iii) 매끄러운 혼합 (smooth mixture) — 실제 Yaad 에 쓰이는 형태. 메모리가 들어오는 데이터에 ℓ_2 와 ℓ_1 중 어느 것이 나을지정한다:

$$W_t = W_{t-1} - \begin{cases} \eta_t \nabla \ell_2(W_{t-1}; k_t, v_t) & \|\mathcal{M}(k_t) - v_t\| \leq \delta_t, \\ \eta_t \delta_t \nabla \ell_1(W_{t-1}; k_t, v_t) & \text{그 외.} \end{cases}$$

δ_t 의 역할은 위와 같으며 입력 의존적으로 학습된다 — 메모리 스스로 토큰마다 그 사건이 크기를 감쇄해야 할 이상치인지 판단한다.

변형 3: 값 이동 (value shift) 에 견고한 메모리. 값 파라미터의 작은 이동에 강인한 메모리를 설계하기 위해 견고 최적화 (robust optimization)를 쓴다. 예측 출력과 섭동된 참값 사이의 최악의 경우 ℓ_2 거리를 손실로 정의한다:

$$L(\mathcal{M}(W, k_t); v_t) = \max_{\|\delta_{v_t}\|_2 \leq \Delta} \frac{1}{2} \|\mathcal{M}(W, k_t) - (v_t + \delta_{v_t})\|_2^2.$$

내부 최대화의 최적 섭동은 닫힌 형태로:

$$\delta_{v_t}^* = \Delta \frac{-\mathcal{M}(W, k_t) + v_t}{\|\mathcal{M}(W, k_t) - v_t\|_2}.$$

이를 대입하면 견고 손실은:

$$L(\mathcal{M}(W, k_t); v_t) = \frac{1}{2} \|\mathcal{M}(W, k_t) - v_t\|_2^2 + \Delta \|\mathcal{M}(W, k_t) - v_t\|_2 + \frac{1}{2} \Delta^2,$$

즉 표준 ℓ_2 손실과 오차의 ℓ_2 노름에 비례하는 항의 결합이며 Δ 가 그 절충을 제어한다. 상수 Δ , Euclidean retention $\text{Ret}_t = \|W - W_{t-1}\|^2$, 선형 메모리 가정 하에 갱신은:

$$W_t = W_{t-1} - \eta \left((\mathcal{M}(W_{t-1}, k_t) - v_t) k_t^\top + \Delta \frac{\mathcal{M}(W_{t-1}, k_t) - v_t}{\|\mathcal{M}(W_{t-1}, k_t) - v_t\|_2} k_t^\top \right).$$

Δ 는 학습 파라미터로 두어 모델이 관찰한 데이터에 따라 견고성을 조절하게 할 수 있다.

5.2 대안적 Retention Gate

변형 1: f -Divergence를 통한 확률 심플렉스 위의 기억. 수치적불안정과 값 폭발을 막는 흔한 기법은 탐색 공간을 유계 영역으로 제한하는것이다. 이에 따라 상태 W_t 를 (스케일된) 확률 심플렉스에 두도록제약한다:

$$\mathcal{W} = \{W \mid \|W\|_1 = c, W_{jl} \geq 0, \forall j, l\}.$$

이 집합에서 각 W 는 하나의 measure 로 볼 수 있으므로, (Learning-Retaining Viewpoint)에서 measure 위의 divergence 를 premetric 으로 쓸 수 있다. $f(1) = 0$ 인 매끄러운 강볼록 함수 $f: \mathbb{R}_+ \rightarrow \mathbb{R}$ 에 대해 f -divergence 를:

$$D_t(W, W') = \sum_{jl} W'_{jl} f\left(\frac{W_{jl}}{W'_{jl}}\right)$$

로 정의한다. 이 premetric 을 retention gate 로, $\tilde{\ell}(W; k_t, v_t) = \langle W - W_{t-1}, \nabla \ell(W_{t-1}; k_t, v_t) \rangle$ 로두면 갱신은:

$$W_t = W_{t-1} \odot g(-\zeta_t - \eta_t \nabla \ell(W_{t-1}; k_t, v_t)).$$

여기서 $g = (f')^{-1}$ 이고 ζ_t 은 $\|W_t\|_1 = c$ 이 되도록정한다. 이 식은, f 가 강볼록 · 매끄러워 g 가 잘정의됨을 뜻한다. 역으로 임의의 강단조 g 에 대해 $f(\tau) = \text{const} + \int_{\tau'=0}^{\infty} g^{-1}(\tau') d\tau'$ 로정의할 수 있어, 연속 단조 증가 함수 g 를 직접 골라 위 갱신을 써도된다.

KL divergence 특수화. $f(\tau) = \tau \ln \tau$ 로 두면 f -divergence 는 KL divergence $D_t(W, W_{t-1}) = \sum_{jl} W_{jl} \log \frac{W_{jl}}{(W_{t-1})_{jl}}$ 이된다. 나아가 균등 분포로부터의 이탈을 벌하는 Shannon 엔트로피를 전역 retention $G_t(W) = \sum_{jl} W_{jl} \log(W_{jl})$ 으로 쓸 수 있다. 국소 · 전역 retention 을 결합하고 KKT 조건을 적용하면 갱신은:

$$W_t \leftarrow c \text{Softmax}((1 - \lambda_t) \log(W_{t-1}) - \eta'_t \nabla \ell(W_{t-1}; k_t, v_t)), \quad \lambda_t = \frac{1/\alpha_t}{1/\alpha_t + 1/\eta_t}, \quad \eta'_t = \frac{1}{1/\alpha_t + 1/\eta_t}.$$

$\lambda_t \in (0, 1)$ 과 $\eta' \in \mathbb{R}_+$ 은 학습 가능하다. 이식은 **exponentiated-gradient / multiplicative-weights** 를 메모리 규칙으로 쓴 것으로, Softmax 가 상태항목을 양수 · 합-정규화로 유지하여 문맥 길이에 관계없이상태가 폭발하지 못하게 한다 — 감쇠 (decay) 가 아니라**재정규화 (renormalization)**에 의한 retention 이다. 위 계산은 행렬 W 에 대한 것이지만, W 가 신경망 (또는 slice 별로 정규화) 일 때도비슷한 갱신이 성립한다.

변형 2: Elastic Net 정규화 — Hard 와 Soft Forgetting. Elastic net 은 LASSO 의 특징 선택 능력과 Ridge 의 편향 감소 성질을 절충하는 회귀도구다. (Learning-Retaining Viewpoint)에서 $\tilde{\ell}_t(W; k_t, v_t) = \langle W - W_{t-1}, \nabla \ell(W_{t-1}; k_t, v_t) \rangle$ 과전역 retention $G_t(W) = \frac{1}{2\beta} \|W\|_2^2 + \frac{1}{\alpha} \|W\|_1$, 국소 $D_t = \frac{1}{2} \|W - W_{t-1}\|_2^2$ 을 두면 갱신은:

$$W_t = \mathcal{S}_\gamma(\lambda W_{t-1} - \zeta \nabla \ell(W_{t-1}; k_t, v_t)), \quad \gamma = \frac{\eta \beta}{\alpha(\eta + \beta)}, \quad \lambda = \frac{\beta}{\beta + \eta}, \quad \zeta = \eta \lambda,$$

여기서 $\mathcal{S}_\gamma(z) = \text{sign}(z) \max\{0, |z| - \gamma\}$ 은 원소별 soft-thresholding 이다. 큰 z 에 대해 z 를 γ 만큼 0 에가깝게 만들고, 이미 0 의 γ -근방이면 정확히 0 으로만든다 (**hard forget**). 이 식은 **soft forgetting**(W 에 $\lambda \in (0, 1)$ 을 곱하는 승법 감쇠, 고전적 게이트) 과 **hard forgetting**(γ 보다 작으면 0 으로 스냅해 용량 확보) 의 결합으로 볼수 있다. γ, λ, ζ 은 학습 가능하다. shrinkage 연산자는 미분 불가능하므로 $\mathcal{S}_\gamma(z) \approx \frac{|z| \arctan(z/\gamma)}{\pi/2}$ 로 매끄럽게 근사한다.

변형 3: Elastic Net 정규화 – Soft-thresholding 을 통한 Forgetting (FTRL). elastic net 은 (FTRL Viewpoint) 에서도 쓸 수 있다. $\frac{1}{\eta_t} R_t(W) = \frac{1}{\eta} \|W\|^2 + \frac{1}{\alpha} \|W\|_1$ 과 $\hat{\ell}(W; x_i) = \langle W - W_{i-1}, \nabla \ell(W_{i-1}; x_i) \rangle$, $W_0 = 0$ 을 두면:

$$A_t = A_{t-1} - \eta \nabla \ell(W_{t-1}; k_t, v_t), \quad W_t = \mathcal{S}_{\eta/\alpha}(A_t).$$

$\mathcal{S}_{\eta/\alpha}$ 은 파라미터 η/α 의 soft-thresholding 으로 변형 1 처럼 매끄럽게 근사한다.

변형 4: 일반 L_q 메모리 안정성. 기존 연구는 $D_t = \|W - W_{t-1}\|_F^2$ 또는 $R(W) = \|W\|_2^2$ 을 쓰지만, 다른 선택도 가능하다. (FTRL Viewpoint) 에서 $1 < q \leq 2$ 에 대해 $\frac{1}{\eta_t} R(W) = \frac{1}{2\eta(q-1)} \|W\|_q^2$ 로 두고 $\hat{\ell}_i(W; k_t, v_t) = \langle W - W_{i-1}, \nabla \ell(W_{i-1}; k_t, v_t) \rangle$ 을 쓰면 (Shalev-Shwartz et al. 2012, 2.6 절) 갱신은 $W_t = A_t / \|A_t\|_p^{p-2}$, $p = \frac{q}{q-1}$ (쌍대 노름) 이며 재귀적으로:

$$A_t = A_{t-1} - \eta \nabla \ell(W_{t-1}; k_t, v_t), \quad W_t = \frac{A_t}{\|A_t\|_p^{p-2}}.$$

이 식은 메모리가 원 (raw) 그래디언트 누적기 A_t 을 유지하고, 노름이 제어된 뷰 W_t 을 노출함을 뜻한다.

변형 5: Retention Gate로서의 Bregman Divergence. Bregman divergence 를 retention gate 로 쓰면 mirror-descent 계열 알고리즘이 나온다. 매끄러운 강볼록 $f : \mathbb{R} \rightarrow \mathbb{R}$ 에 대해 $F(W) = \sum_{j,l} f(W_{jl})$, $D_t(W, W') = F(W) - F(W') - \langle \nabla F(W'), W - W' \rangle$ 로 두고 $\tilde{\ell}_t = \langle W - W_{t-1}, \nabla \ell(W_{t-1}; k_t, v_t) \rangle$ 을 쓰면:

$$W_t = g(-\eta \nabla \ell(W_{t-1}; k_t, v_t) + F'(W_{t-1})),$$

여기서 F' 은 f' 을 원소별로 적용한 것, $g = (F')^{-1}$ 이다. $f(\tau) = \tau^2/2$ 이면 F' 과 g 가 항등이 되어 갱신은 비선형성 없는 단순 경사하강이 된다. 다른 f 를 고르면 g 에 비선형성이 도입되어 메모리 표현력이 커진다. 예컨대 $f'(\tau) = \ln \frac{\tau}{1-\tau}$ (역 시그모이드) 로 두면 $g = \sigma$ (시그모이드) 가 되어:

$$W_t = \sigma\left(\ln \frac{W_{t-1}}{1 - W_{t-1}} - \eta \nabla \ell(W_{t-1}; k_t, v_t)\right),$$

이 식은 모든 상태 항목을 $(0, 1)$ 로 가둔다 – retention gate 가 곧 비선형성이 되어 순환 자체에 표현력을 주입한다.

5.3 Miras의 변형: Moneta, Yaad, Memora

앞 절에서 attentional bias와 retention gate의 다양한 선택으로 Miras의 일반성을 보였다. 이제 프레임워크 위에서 서로 다른 동기로 설계된 세 새로운 시퀀스 모델을 제시하고, 이들이 어떻게 빠른 병렬 학습을 활용하는지 논한다. 세 모델 모두 메모리 아키텍처로 **확장 인자 4, GELU 활성화, 잔차 연결과 layer norm을 갖춘 2-layer MLP** $\mathcal{M}(x) = x + \text{LN}(W_1 \sigma(W_2 x))$ 을 쓰고, 메모리 알고리즘으로 **단순 경사하강**을 쓴다.

Moneta. $p, q \in \mathbb{R}_{\geq 1}$ 에 대해 (p, q) -Moneta를 다음으로 설계한다. (1) 메모리는 위 2-layer MLP. (2) attentional bias는 ℓ_p . (3) retention gate는 ℓ_q retention $\frac{1}{2(q-1)}\|W\|_q^2$ 과 표준 ℓ_2 정규화 $\frac{1}{\beta}\|W\|_2^2$ 의 하이브리드. (4) memory 알고리즘은 경사하강. 결과 순환식:

$$A_t = \alpha_t A_{t-1} - \eta_t \nabla \ell_p(W_{t-1}; k_t, v_t), \quad W_t = \frac{A_t}{\|A_t\|_q^{q-2}},$$

여기서 그래디언트는 $\nabla \ell_p(W_{t-1}; k_t, v_t) = p[\text{Sign}(W k_t - v_t) \odot |W k_t - v_t|^{p-1}] k_t^\top$ 이다. 우리는 $(p, q) = (3, 4)$ 을 쓴다. 설계 취지: $p = 3$ 은 잘못된 recall된 (놀라운) 토큰에 대한 주의를 날카롭게 하고, ℓ_q 정규화는 노출 메모리를 노름이 제어된 껍질 위에 두어 승법 감쇠만보다 강한 안정화를 준다.

Yaad. 극단적 사건 (토큰) 으로부터 자신을 보호하는 견고 메모리의 중요성에 기반해 Yaad를 Huber 목적함수로 설계한다. 메모리는 위와 같은 2-layer MLP, attentional bias는 Huber 손실 (5.1 절 변형 2-(iii)), retention gate는 국소·전역의 결합 $\text{Ret}_t(W, W_{t-1}) = \frac{1}{2\theta_t}\|W - W_{t-1}\|_F^2 + \frac{1}{\beta_t}\|W\|_2^2$ (Behrouz et al. 2024c의 “forget gate”와 동치), 알고리즘은 경사하강. 결과:

$$W_t = \alpha_t W_{t-1} - \begin{cases} \eta_t \nabla \ell_2(W_{t-1}; k_t, v_t) & \|\mathcal{M}(k_t) - v_t\| \leq \delta_t, \\ \eta_t \delta_t \nabla \ell_1(W_{t-1}; k_t, v_t) & \text{그 외.} \end{cases}$$

표현력을 위해 모든 아키텍처에서 학습률 η 과 retention gate 비율 α 을 분리 (decouple) 하여 독립 파라미터 $\beta_t \in [0, 1]^d$ 을 둔다.

Memora. elastic net 정규화 (hard·soft retention)의 발상을 쓴다. (1) 메모리는 위와 같은 2-layer MLP, (2) attentional bias는 단순 ℓ_2 회귀, (3) retention gate는 KL divergence (식 21), (4) 알고리즘은 경사하강. 결과:

$$W_t = \text{Softmax}(\alpha_t \log(W_{t-1}) - \eta_t \nabla \ell_2(W_{t-1}; k_t, v_t)).$$

취지: 심플렉스 제약 (softmax 재정규화) 상태는 유계임이 보장되어, forgetting이 감쇠가 아니라 항목 간 확률질량 경쟁에서 발생하며 문맥 길이에 관계없이 상태가 폭발하지 않는다.

5.4 아키텍처 백본과 빠른 학습

아키텍처 백본. 최근 연구 (Behrouz et al. 2024c; Yang et al. 2024a)를 그대로 따른다. Llama의 매크로 아키텍처에서 attention 모듈을 Miras 변형으로 교체하고, SwiGLU MLP, RoPE (Su et al. 2024), RMSNorm (Zhang et al. 2019)을 쓴다. Miras layer 블록에서는 query·key·value 투영 각각 뒤에 1D depthwise-separable 컨볼루션 (커널 크기 4)을 두고, 학습 안정성을 위해 q와 k에 ℓ_2 정규화를 적용한다. Miras layer 블록의 출력은 정규화 후선형 층으로 게이팅 (output gate, Mehta et al. 2023)된다.

채널별 파라미터. 학습 파라미터 η_t, δ_t 와 retention gate α_t 는 채널별로 ($\eta_t, \delta_t, \alpha_t \in \mathbb{R}^d$) 파라미터화한다. 표현력은 커지지만 파라미터가 크게 늘어나므로, Peng et al. (2025b)을 따라 입력을

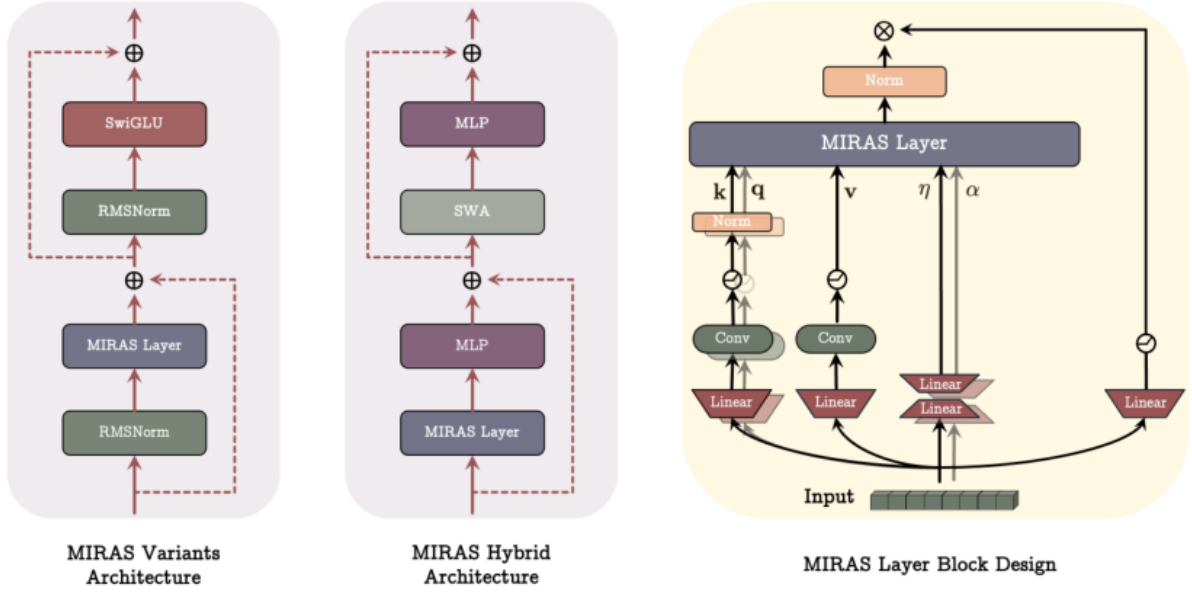


그림 2: Miras 변형 아키텍처, SWA와 결합한 하이브리드 버전, 그리고 Miras layer의 블록 설계 시각화. 출처: 원저자, arXiv:2504.13173 Fig.2

$\mathbb{R}^k$ 로, 다시 $\mathbb{R}^d$ 로 투영하는 저랭크 (low-rank) 투영을 쓴다 (k 는 보통 32 또는 64). 백본은 그림 2에 있다.

하이브리드 모델. 하이브리드 버전도 평가한다. Samba(Ren et al. 2024) 아키텍처를 따라 Miras layer를 **Sliding Window Attention(SWA)**와 순차적으로 교대 배치한다 (그림 2).

병렬 학습. Miras 변형은 이론적으로 잘 동기화되었지만 순환이비선형이라 대규모에서 직접 학습이 느릴 수 있다. Behrouz et al.(2024c)과 Sun et al.(2024)을 따라 이를 병렬화한다. 핵심 아이디어는 시퀀스를 크기 b (보통 16 또는 64)의 청크로 나누고, 현재 청크 내 모든 토큰의 그래디언트를 **이전 청크의 마지막 상태**에 대해 계산하는 것이다. 즉 $\nabla \ell(\mathcal{M}_{t-1}; k_t, v_t)$ 대신 $\nabla \ell(\mathcal{M}_{t'}; k_t, v_t)$ 을 쓰며 t' 은 이전 청크의 마지막 상태이다.

이 트릭으로 모든 그래디언트를 한꺼번에 계산해 청크 내 순환을 선형으로 만들 수 있다. 가속기를 온전히 활용하려면 이를 행렬곱으로 재구성해야 한다. Moneta에서 $q = 2$ 로 두고 순환을 전개하면:

$$\mathcal{M}_t = \alpha_t \mathcal{M}_{t-1} - \eta_t \nabla \ell(\mathcal{M}_{t-1}; k_t, v_t) = \beta_t \mathcal{M}_0 - \sum_{i=1}^t \frac{\beta_t}{\beta_i} \eta_i \nabla \ell(\mathcal{M}_{t'}; k_i, v_i),$$

여기서 $t' = t - \text{mod}(t, b)$, $\beta_i = \prod_{j=1}^i \alpha_j$ 이다. 첫 청크 ($t = b, t' = 0$)에서 선형 메모리 $\mathcal{M}_t = W_t$ 일 때 ℓ_p 손실에 대해:

$$\nabla \ell(W_0; k_t, v_t) = p \text{Sign}(W_0 k_t - v_t) \odot |W_0 k_t - v_t|^{p-1} k_t^\top \Rightarrow \sum_{i=1}^b \eta_i \frac{\beta_b}{\beta_i} \nabla \ell(W_0; k_i, v_i) = p E_b \odot B_b \odot \text{Sign}(W_0 K - V) \odot |W_0 K - V|^{p-1} K^\top,$$

여기서 $E_b = \text{diag}(\eta_1, \dots, \eta_b)$, B_b 은 β_b/β_i 에 대해 유사하게 정의된다. 학습 안정성을 위해 $\text{Sign}(x) \approx \tanh(\alpha x)$, $|x| \approx \sqrt{x^2 + \epsilon}$ ($\epsilon = 10^{-6}$)을 쓴다. $q \neq 2$ 인 경우는 메모리에 대한 정규화 항으로 나타나며, Titans-TTT처럼 이 비선형성을 청크 안에서가 아니라 **청크 끝에서만** 적용한다.

Yaad는 위와 매우 유사하며, ℓ_1 과 ℓ_2 손실의 그래디언트를 모두 계산하고 $\|\mathcal{M}(k_t) - v_t\| \leq \delta_t$ 수준의 마스킹으로 결합한다.

Memora는 softmax와 log의 두 비선형 부분이 있어 병렬화가 어렵다. 청크 안에서는 선형 버전을, 청크 간에는 비선형 버전을 쓰되, 청크 끝에서 log와 softmax를 함께 쓰면 log의 효과가

상쇄된다. 이를 피하기 위해 각체크 뒤에 **lag token**(인덱스 $i = kb + 1$) 을 둔다. 이전 체크의 마지막 상태를 $\mathcal{M}_0$ 이라 하면:

$$\mathcal{M}_1 = \text{Softmax}(\alpha_1 \log(\mathcal{M}_0) - \eta_1 \nabla \ell_2(\mathcal{M}_0; k_1, v_1)),$$

이후 체크는 $\mathcal{M}_1$ 부터 선형으로 진행한다. 선형 메모리 $\mathcal{M}_1 = W_1$ 일 때:

$$\nabla \ell(W_1; k_t, v_t) = (W_1 k_t - v_t) k_t^\top \Rightarrow \sum_{i=1}^b \eta_i \frac{\beta_b}{\beta_i} \nabla \ell(W_1; k_i, v_i) = E_b \odot B_b \odot (W_1 K - V) K^\top,$$

행렬 정의는 위와 같다.

6 실험 (Experiments)

실험에서 우리는 세 질문에 답하고자 한다. (1) 서로 다른 attentional bias 가 실제로 다른 아키텍처를 낳는가? (2) 서로 다른 유형의 retention gate 가 긴 문맥에서 성능에 어떻게 영향을 주는가? (3) Memora, Moneta, Yaad 는 다운스트림 과제에서 베이스라인 대비 어떤가?

설정. 학습 문맥 창 4096 으로, LM·상식 추론에는 FineWeb-Edu(Penedo et al. 2024) 를, 스케일링 패턴에는 C4(Raffel et al. 2020) 를 쓴다. 모델 크기는 120M, 340M, 760M, 1.3B. 소형 (120M·340M) 은 15B 토큰, 중형 (760M) 은 30B, 대형 (1.3B) 은 100B 토큰으로 학습한다. 베이스라인 결과는 Behrouz et al.(2024c) 에서 가져왔다.

6.1 언어 모델링과 상식추론

먼저 언어 모델링 perplexity 와 상식 추론 과제에 집중한다. 340M·760M·1.3B 에 대한 결과는 표 2 에 있다. **우리의 모든 변형이 Transformer++, 현대 linear RNN, 하이브리드방법을 포함한 모든 베이스라인을 능가한다.** 특히 우리 변형은 모두 순수순환 (attention-free) 임에도 하이브리드 모델보다 우수하다는 점이 중요하다. 세 변형 중 Moneta 가 Memora·Yaad 보다 약간 약하나 서로 근접하며, 최고 모델은 과제·크기에 따라 달라진다.

표 2: Miras 변형과 베이스라인의 언어 모델링·상식추론 성능. 하이브리드 모델은 *로 표시. 단순·하이브리드각각의 최고를 굵게. 최대 규모에서는 우리의 단순 모델을 하이브리드와 비교한다.

모델	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA ↑	Hella. ↑	Wino. ↑	ARC-e ↑	ARC-c ↑	SIQA ↑	BoolQ ↑
340M params / 15B to- kens										
Trans- former++	31.52	41.08	30.76	62.98	34.76	50.53	45.21	24.05	36.81	58.24
Ret- Net	32.50	49.73	28.24	62.61	34.15	50.91	44.27	23.62	36.79	59.72
GLA	28.51	43.02	28.73	64.05	35.96	50.00	54.19	24.29	37.13	58.39
Mamba	30.83	40.21	29.94	63.79	35.88	49.82	49.24	24.56	35.41	60.07
DeltaNet	28.65	47.30	28.43	63.52	35.95	49.63	52.68	25.37	37.96	58.79
TTT	27.44	34.19	30.06	63.97	35.71	50.08	53.01	26.11	37.32	59.83
Gated DeltaNet	27.01	30.94	34.11	63.08	38.12	51.60	55.28	26.77	34.89	59.54

모델	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA ↑	Hella. ↑	Wino. ↑	ARC-e ↑	ARC-c ↑	SIQA ↑	BoolQ ↑
Mon-eta (ours)	26.19	29.31	35.70	63.99	39.23	52.04	55.96	27.15	37.29	60.22
Yaad (ours)	26.61	29.11	34.09	64.93	39.86	51.12	54.75	28.64	33.82	60.29
Mem-ora (ours)	27.16	30.44	33.68	65.21	39.17	51.23	53.40	27.99	34.10	59.29
760M params / 30B to- kens										
Trans- former++	25.21	27.64	35.78	66.92	42.19	51.95	60.38	32.46	39.51	60.37
Ret- Net	26.08	24.45	34.51	67.19	41.63	52.09	63.17	32.78	38.36	57.92
Mamba	22.94	28.37	33.54	67.90	42.71	49.77	63.48	31.09	40.06	58.15
DeltaNet	24.37	24.60	37.06	66.93	41.98	50.65	64.87	31.39	39.88	59.02
TTT	24.17	23.51	34.74	67.25	43.92	50.99	64.53	33.81	40.16	59.58
Gated DeltaNet	21.18	22.09	35.54	68.01	44.95	50.73	66.87	33.09	39.21	59.14
Samba*	20.63	22.71	39.72	69.19	47.35	52.01	66.92	33.20	38.98	61.24
Gated DeltaNet- H2*	19.88	20.83	39.18	68.95	48.22	52.57	67.01	35.49	39.39	61.11
Mon-eta (ours)	21.18	21.94	38.02	69.55	49.16	53.01	67.47	36.09	40.53	63.18
Yaad (ours)	20.99	21.57	37.85	69.14	50.02	53.93	67.78	36.27	41.01	63.34
Mem-ora (ours)	22.28	22.31	38.19	67.82	49.30	53.28	63.57	36.15	40.94	62.96
Moneta-H (ours)	18.72	20.13	40.59	70.84	50.13	54.17	67.64	36.79	40.87	62.43
Yaad-H (ours)	18.59	19.80	40.22	69.51	50.48	53.69	68.04	36.55	40.28	61.94
Memora-H (ours)	18.24	20.55	39.91	69.06	49.84	52.88	66.90	36.12	40.99	61.75
1.3B params / 100B to- kens										
Trans- former++	18.53	18.32	42.60	70.02	50.23	53.51	68.83	35.10	40.66	57.09
Ret- Net	19.08	17.27	40.52	70.07	49.16	54.14	67.34	33.78	40.78	60.39
Mamba	216.56	12.56	45.66	71.87	55.67	55.24	72.47	37.88	40.20	60.13
DeltaNet	17.71	16.88	42.46	70.72	50.93	53.35	68.47	35.66	40.22	55.29
Gated DeltaNet	16.42	12.17	46.65	72.25	55.76	57.45	71.21	38.39	40.63	60.24

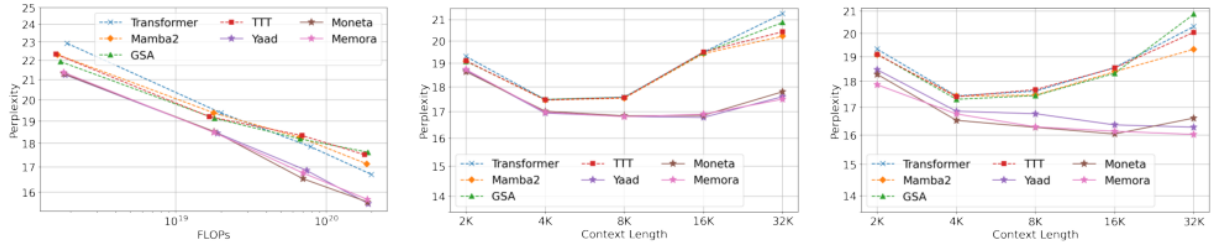


그림 3: C4 데이터셋에서 (좌) 모델 크기, (중) 시퀀스 길이 (모델 크기 340M), (우) 시퀀스 길이 (모델 크기 760M) 를 늘릴 때의 스케일링 패턴. 출처: 원저자, arXiv:2504.13173 Fig.3

모델	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA ↑	Hella. ↑	Wino. ↑	ARC-e ↑	ARC-c ↑	SIQA ↑	BoolQ ↑
Samba*	16.13	13.29	44.94	70.94	53.42	55.56	68.81	36.17	39.96	62.11
Gated	15.91	12.55	48.76	72.19	56.88	57.77	71.33	39.07	41.91	61.55
DeltaNet-H2*										
Moneta	15.52	11.47	47.88	73.16	56.14	59.09	72.53	40.32	41.91	61.18
(ours)										
Yaad	15.18	11.89	47.23	72.81	56.46	59.02	72.14	40.05	40.73	61.86
(ours)										
Memora	15.90	12.04	48.67	73.10	55.99	57.36	71.55	37.92	40.19	61.34
(ours)										

1.3B/100B 에서 순수 순환 Yaad(15.18) 와 Moneta(15.52) 는 Gated DeltaNet(16.42) 뿐 아니라 하이브리드 Samba(16.13) 와 Gated DeltaNet-H2(15.91) 까지 능가한다.

6.2 스케일링 패턴

문맥 길이. 340M·760M 두 버전에서 학습 문맥 길이를 2K 에서 32K 까지 변화시킨다 (그림 3 중앙·우측). 세 변형 모두 문맥 길이를 늘릴 때 SOTA 베이스라인보다 잘 스케일한다. 이 우수성은 (1) 표현력 있는 메모리 아키텍처 (Mamba2·GSA 의 벡터·행렬 메모리와 달리 2-layer MLP 사용), (2) attentional bias 와 retention gate 의 선택 (고정 크기 용량을 더 잘 관리) 에 기인한다.

모델 크기. #FLOPs 대 perplexity 를 그림 3(좌측) 에 보고한다. 세 변형 모두 거의 같은 FLOPs 예산에서 모든 베이스라인을 능가하여, 강력한 메모리 설계의 중요성을 재확인한다.

6.3 Needle In Haystack

유효 문맥 창을 평가하기 위해 needle-in-haystack 과제를 쓴다. 긴 방해 텍스트 (“haystack”) 에서 정보 조각 (“needle”) 을 회수하는 과제로, RULER 벤치마크 (Hsieh et al. 2024) 의 Single NIAH(S-NIAH) 를 길이 1K·2K·4K·8K 에서 평가한다 (표 3). **우리의 모든 변형이 상당한 격차로 베이스라인을 능가한다.** 흥미롭게도 데이터가 합성 잡음 (S-NIAH-PK) 일 때 Moneta 가 다른 변형보다 우수하는데, 이는 p -노름 목적함수와 retention gate 가 잡음에 더 견고함을 검증한다.

표 3: RULER 벤치마크 NIAH 과제에서 Moneta·Yaad·Memora 와 베이스라인의 성능. 최고 정확도를 굵게.

모델	PK 2K	PK 4K	PK 8K	N 2K	N 4K	N 8K	W 1K	W 2K	W 4K	평균
Mamba	298.6	61.4	31.0	98.4	55.8	14.2	62.2	42.2	4.2	52.0
DeltaNet	96.8	98.8	98.6	47.2	15.4	12.8	85.2	46.2	20.0	57.9

모델	PK 2K	PK 4K	PK 8K	N 2K	N 4K	N 8K	W 1K	W 2K	W 4K	평균
Gated DeltaNet	89.8	91.4	90.0	99.2	91.8	26.4	86.4	82.6	24.4	75.8
TTT	98.4	98.8	98.0	60.2	36.6	10.2	85.8	78.8	28.0	66.1
Mon-eta	99.4	98.8	98.8	99.4	99.4	92.8	92.2	88.2	70.8	93.5
Yaad	99.2	98.6	94.4	99.8	98.6	93.2	91.8	89.6	67.4	92.9
Mem-ora	99.2	98.8	92.6	98.4	99.2	93.2	92.4	88.2	70.4	92.1

(PK = S-NIAH-PK, N = S-NIAH-N, W = S-NIAH-W)

6.4 소거 연구 (Ablation Study)

논문에서 논한 설계 선택들이 실제로 더 나은 결과에 긍정적으로 기여하는지 검증한다.

p 의 효과. Moneta에서 $p \in \{1, 1.5, 2, 2.8, 3, 3.2, 4\}$, 문맥 창 2K~16K를 변화시킨다 (그림 4). 흥미롭게도 p 를 늘릴 때 **단조 패턴이없으며** 최고 성능은 $p = 3$ 에서, 최악은 $p = 4$ 에서 나온다. 또한 p 가 다르면 메모리 모듈 성능도 다르지만, 문맥 길이를 늘릴 때의 스케일링 패턴은 거의 같다.

q 의 효과. $q \in \{2, 3, 4, 5\}$ 을 변화시킨다. p 와 달리 q 는 문맥 길이를 늘릴 때의 **스케일링 패턴을 바꾼다**. q 가 retention gate를 결정하고, 강력한 retention gate가 메모리 관리를 개선해 더 나은 성능을 주기 때문이다.

설계의 효과. Yaad에 대해 소거 연구를 수행한다 (표 4). 각 행은 (1) Yaad 원본, (2) retention 제거 ($\beta = 1$), (3) δ 을 입력비의존으로, (4) Huber에서 ℓ_2 손실 제거, (5) ℓ_1 조건제거, (6) MLP를 선형 층으로 교체한 경우이다. 모든 설계 선택이 성능에 기여함을 확인한다.

표 4: Yaad 구성요소 소거 연구.

모델	Avg. LM
Yaad	53.98
- Retention Gate	50.63
- 입력 의존적 δ	52.19
- ℓ_2 -loss	52.86
- ℓ_1 -loss	53.04
- linear memory	51.57

7 결론 (Conclusion)

이 논문에서 우리는 온라인 최적화와 테스트 시점 기억의 연결을 설명하는 일반 프레임워크 **Miras**를 제시했다. Miras는 문헌의 여러 표준 아키텍처 선택 (예: forget gate)의 역할을 설명하고, 메모리를 더 잘 관리하는 차세대 아키텍처 설계를 돕는다. 이 프레임워크 위에서 각각의 장단점을 지닌 새로운 시퀀스 모델을 제시했으며, 실험은 이 변형들이 다양한 다운스트림 과제에서 Transformer와 linear RNN보다 강력함을 보였다. 앞으로 이 대안 아키텍처들을 여러 다운스트림 과제에서 탐구하는 것은 흥미로운 방향이다.

부록 A: 추가 관련 연구 (Additional Related Work)

현대 Linear RNN. Transformer의 제곱 비용과 긴 문맥 한계를 극복하려는 효율적 순환 대안이 활발히 연구되었다 (Tiezzi et al. 2024). 첫세대 (RetNet, LRU, RWKV, S5, S4)는 데이터 비의존 전이 행렬과 Hebbian 갱신을 쓴다. 둘째 세대는 입력 의존 파라미터를 도입하거나 (Griffin, SSMs,

Mamba2	98.6	61.4	31.0	98.4	55.8	14.2	62.2	42.2	4.2	52.0
DeltaNet	96.8	98.8	98.6	47.2	15.4	12.8	85.2	46.2	20.0	57.9
Gated DeltaNet	89.8	91.4	90.0	99.2	91.8	26.4	86.4	82.6	24.4	75.8
TTT	98.4	98.8	98.0	60.2	36.6	10.2	85.8	78.8	28.0	66.1
MONETA	99.4	98.8	98.8	99.4	99.4	92.8	92.2	88.2	70.8	93.5
YAAD	99.2	98.6	94.4	99.8	98.6	93.2	91.8	89.6	67.4	92.9
MEMORA	99.2	98.8	92.6	98.4	99.2	93.2	92.4	88.2	70.4	92.1

achieves the worst performance. Also, although different values of p results in different memory performance, the scaling pattern when increasing the context length is almost the same.

The Effect of q on Performance. Similarly, we evaluate the effect of q by varying it in $\{2, 3, 4, 5\}$. In addition to p , the value of q can change the scaling pattern when increasing the context length. The main reason is that the value of q determines the retention gate and a powerful retention gate can improve the performance, resulting in better performance.

The Effect of Design. To evaluate the architectural design choices, we perform an ablation study. The results are in Table 4. The first row, reports the performance of YAAD, while (1) the second row removes the retention gate, (2) the third row makes δ input independent, (3) the fourth row removes ℓ_2 -loss from the Huber loss, (4) the fifth row removes the ℓ_1 condition, and (5) the last row replaces the MLP with a linear layer. These results indicate that the retention gate is contributing to the performance of the model.

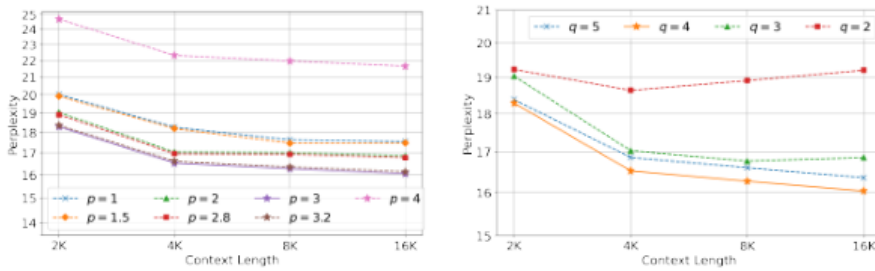


Table 4: Ablation results of YAAD.

Model
YAAD
- Retention G
- Input-depen

그림 4: 서로 다른 문맥 길이에서 파라미터 p 와 q 가 성능에 미치는 효과. 출처: 원저자, arXiv:2504.13173 Fig.4

RWKV6) delta 규칙 기반의 더 표현력 있는 갱신을 쓴다. 다음 세대는 메모리를 깊은 모델로 확장하며 delta 규칙 (Sun et al. 2024) 또는 모멘텀 기반 갱신 (Behrouz et al. 2024c) 을 쓴다. 최근 Siems et al.(2025) 은 토큰당다중 경사하강으로 state-tracking 과제에서 더 표현력 있는 모델을 제안했다. 이 밖에도 (해석 가능한) 비선형 RNN 연구가 있으나, 순환적 성질 탓에 대규모 병렬화는 여전히 도전적이다.

Fast Weight Programs. 선형 층을 키-값 associative memory 로 해석하는 발상은 Hopfield 네트워크 (Hopfield 1982) 와 fast weight program 으로 거슬러 올라가며, 동적 fast program 을 쓰기 가능한 메모리로 RNN 에 통합한다 (Schlag et al. 2021; Schmidhuber 1992, 1993). Hebbian (Hebb 2005) 과 delta 규칙 (Prados et al. 1989) 이 가장 널리 쓰인 학습 규칙이다.

Test Time Training. 테스트 시점 학습의 발상은 초기 local learning (Bottou et al. 1992) 으로 거슬러 올라가며, 각 테스트 데이터에 예측 전에 그 이웃으로 학습한다. 현대 아키텍처에 적용되어 비전 · 비디오 생성 등에서 out-of-distribution 완화 능력 덕에 유망한 성능을 보였다.

Hopfield Networks. 우리는 associative memory 의 광의 형태 위에 Miras 를 세운다. Hopfield 네트워크 (Hopfield 1982) 는 associative memory 를 키 · 값 저장에 필요한 에너지 함수의 최소화로 정의한 초기 연구이다. 전통 Hopfield 네트워크는 벡터 메모리 · 에너지 함수의 용량 한계로 적용성이 제한적이었으나, 최근 여러 연구가 지수 커널로 에너지 함수를 확장하는 등으로 용량을 개선하고 Transformer 와의 연결을 논한다.

통합 프레임워크. 시퀀스 모델의 밑바탕 메커니즘을 이해하고 (일부를) 하나의 관점으로 통합하려는 노력이 늘고 있다. Dao et al.(2024) 의 **SSD** 프레임워크는 결합 연산자와 구조화 행렬로 선형 Transformer 와 (일부) linear RNN 을 연결하나, 벡터 · 행렬 메모리와 Hebbian 갱신에 국한된다. Liu et al.(2024a) 은 (일부) linear RNN 에 온라인 학습관점을 제시했으나 확률적 optimizer 를 쓰는 행렬 메모리 온라인 학습자에 국한되어 Transformer · Titans 는 표현할 수 없다. Sun et al.(2024) 은 선형 · softmax attention 이 각각 (커널) 회귀의 파라메트릭 · 비파라메트릭 해임을 보였으나, 목적 함수를 회귀로 제한하여 다른 linear RNN 은 클래스 밖에 둔다. 동시기 Wang et al.(2025) 도 모델을 같은 attentional bias 로 강제하여, 현대 RNN (RetNet, Mamba) 을 추가단 순화하면 회귀를 최적화하는 같은 클래스에 근사적으로 놓임을 보였다. 그러나 이 단순화는 밑바탕 갱신 규칙의 이해를 완전히 바꾼다. Wang et al.(2025) 과 달리 **Miras** 는 **Hebbian 갱신 (내적 유사도)** 과 **delta 규칙 갱신 (회귀 손실)** 모델을 구별할 수 있으며, 이 논문의 모든 모델 (Moneta, Memora, Yaad) 과 HGRN2 (Qin et al. 2024) 는 서로 다른 attentional bias 덕에 그 클래스 밖에 놓인다.

부록 B: 명제 3.2 의 증명 (Proof of Proposition 3.2)

명제 3.2 (재진술). $\eta_t = \eta$ 이라 하고 $h_t(W) := \sum_{i=1}^{t-1} \hat{\ell}_i(W; k_i, v_i) + \frac{1}{\eta} R(W)$ 로 정의하자. $\mathcal{W} = \mathbb{R}^d$ 이고 $h_t(W)$ 가 강볼록이며, D_h 을 h 가 정의하는 Bregman divergence 라 하자. (Learning-Retaining Viewpoint) 에서 $\text{Ret}_t = D_h$, $\tilde{\ell}_t = \hat{\ell}_t$ 로 두면, (Learning-Retaining Viewpoint) 의 갱신은 (FTRL Viewpoint) 의 갱신과 동치이다.

증명 (요지). (FTRL Viewpoint) 으로 얻은 파라미터열 $\{\hat{W}_1, \hat{W}_2, \dots\}$ 과 (Learning-Retaining Viewpoint) 으로 얻은 열 $\{\tilde{W}_1, \tilde{W}_2, \dots\}$ 이 같은 점에서 초기화되면서도 같음을 **귀납법**으로 보인다. 같은 점에서 초기화되므로 기저 ($\tilde{W}_1 = \hat{W}_1$) 가 성립한다. 귀납 가정 $\tilde{W}_{t-1} = \hat{W}_{t-1}$ 하에서, (Learning-Retaining Viewpoint) 의 정의로부터

$$\tilde{W}_t = \arg \min_W \left\langle \sum_{i=1}^{t-1} \nabla \hat{\ell}_i(\hat{W}_{t-1}; k_i, v_i) + \frac{1}{\eta} \nabla R(\hat{W}_{t-1}), W - \hat{W}_{t-1} \right\rangle + \left[(\text{FTRL 목적함수의 나머지}) \right].$$

한편 (FTRL Viewpoint) 에서 $\hat{W}_{t-1} = \arg \min_W \sum_{i=1}^{t-1} \hat{\ell}_i(W; k_i, v_i) + \frac{1}{\eta} R(W)$ 이므로 1 차 최적성 조건

$$\sum_{i=1}^{t-1} \nabla \hat{\ell}_i(\hat{W}_{t-1}; k_i, v_i) + \frac{1}{\eta} \nabla R(\hat{W}_{t-1}) = 0$$

이 성립한다. 이 두 식을 결합하면

$$\tilde{W}_t = \arg \min_W \sum_{i=1}^t \hat{\ell}_i(W; k_i, v_i) + \frac{1}{\eta} R(W),$$

즉 $\tilde{W}_t = \hat{W}_t$ 이 되어 증명이 완결된다. $\square$

부록 C: 실험 설정 (Experimental Setup)

언어 모델링 (Merity et al. 2017; Paperno et al. 2016), 상식 추론 (Bisk et al. 2020; Clark et al. 2019, 2018; Sakaguchi et al. 2021; Zellers et al. 2019), 긴 문맥 needle-in-haystack(Hsieh et al. 2024) 에서 평가한다. 비교 대상은 SOTA linear RNN, Transformer, 하이브리드 (순환+attention) 로, 구체적으로 Transformer++(Touvron et al. 2023), RetNet(Sun et al. 2023), GLA(Yang et al. 2024b), Mamba(Gu et al. 2024), Mamba2(Dao et al. 2024), DeltaNet(Yang et al. 2024c), TTT(Sun et al. 2024), Gated DeltaNet(Yang et al. 2024a) 이다.

표 5: 아키텍처 세부.

모델	Block	Dim	Head	Peak LR	Token
170M	12	768	16	3e-3	15B
340M	24	1024	16	1.5e-3	15B
780M	24	1536	16	1.25e-3	30B